

Explication du code du projet VAS SMS+ Monitor

Ce document explique la structure du projet **Laravel** situe dans `D:\vas_project`. Il presente les fichiers importants, le cheminement d'une requete, l'authentification, les roles, les CRUD, la base de donnees et les points techniques a surveiller.

Type de projet : application web Laravel pour le suivi VAS/SMS+

Framework : Laravel 13 avec PHP 8.3

Base cible : Oracle via `yajra/laravel-oci8`

Interface : Blade + Bootstrap + Chart.js, avec Vite/Tailwind presents dans le projet

1. Idee generale du projet

Le projet est une plateforme de monitoring des revenus et services VAS/SMS+. L'utilisateur se connecte, choisit ou utilise un role, puis accede a un tableau de bord adapte. Les roles principaux sont :

- **Business** : consultation des revenus, analyses, recherche MSISDN et alertes.
- **Operationnel** : suivi CDR, chargement, aggregation, gestion des services et fournisseurs.
- **Admin** : gestion des utilisateurs et modules d'administration.

Plusieurs pages sont encore des placeholders. Les CRUD utilisateurs, services et fournisseurs sont deja implementes.

2. Structure des dossiers importants

Chemin	Role dans le projet
<code>routes/web.php</code>	Definit les routes de l'application : login, dashboard, Business, Operations et Admin.
<code>app/Http/Controllers</code>	Contient la logique qui recoit les requetes et retourne les vues ou redirections.
<code>app/Http/Middleware</code>	Controle l'accès aux pages selon l'authentification et le role.
<code>app/Models</code>	Represente les tables de base de donnees avec Eloquent.
<code>resources/views</code>	Contient les vues Blade responsables du HTML.
<code>app/Console/Commands</code>	Contient les commandes Artisan pour importer les fichiers CDR MMG et OCC.
<code>config/database.php</code>	Contient la configuration des connexions de base de donnees, dont Oracle.
<code>tests</code>	Contient les tests PHPUnit actuels.

3. Cheminement d'une requete Laravel

Navigateur demande une URL comme `/dashboard` -> Laravel cherche la route dans `routes/web.php` -> la requete passe par les middleware, par exemple `dashboard.access` -> elle arrive dans un controller, par exemple `DashboardController@index` -> le controller prepare les donnees -> il retourne une vue Blade, par exemple `dashboard.index` -> le navigateur affiche la page.

4. Le fichier routes/web.php

Ce fichier est la carte de navigation de l'application. Il contient notamment :

- La route `/` qui redirige vers `/login`.
- Les routes de connexion et de reinitialisation du mot de passe.
- Un groupe protege par le middleware `dashboard.access`.
- Des sous-groupes proteges par role : `business`, `operationnel` et `admin`.

```
Route::middleware('dashboard.access')->group(function (): void {  
    Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');
```

```
Route::middleware('role:operationnel')
    ->prefix('operations')
    ->name('operations.')
    ->group(function (): void {
        Route::resource('services', ServiceController::class)->except(['show']);
    });
});
```

Dans cet exemple, `/dashboard` exige une session autorisée. Les routes `operations/services` exigent le rôle `operationnel`.

5. Authentification et rôles

AuthController

Le fichier `app/Http/Controllers/AuthController.php` gère :

- L'affichage de la page de connexion.
- La validation de l'email, du mot de passe et du rôle choisi.
- La vérification du mot de passe.
- La conversion automatique d'un ancien mot de passe en clair vers un hash Laravel, si la connexion réussit.
- Le stockage du rôle actif en session avec la clé `active_role`.
- Le mode démonstration sans Oracle via `AUTH_DEMO_LOGIN=true`.
- La réinitialisation personnalisée du mot de passe avec email et téléphone.

EnsureDashboardAccess

Ce middleware laisse passer la requête si l'utilisateur est connecté avec Laravel Auth ou si la session de démonstration est active. Sinon, l'utilisateur est redirigé vers la page login.

EnsureUserHasRole

Ce middleware lit le rôle actif depuis la session ou depuis l'utilisateur. Il vérifie ensuite que ce rôle est autorisé pour la route demandée. En cas d'échec, il retourne une erreur 403.

6. Le modèle User

Le fichier `app/Models/User.php` représente la table `users`. Il contient les constantes de rôles et les méthodes de conversion entre les rôles utilisés dans l'application et ceux stockés dans Oracle.

Dans l'application	Dans Oracle	Signification
admin	ADMIN	Administrateur
business	BUSINESS	Analyste Business
operationnel	OPERATION	Analyste Operationnel

La methode `normalizeRole` est importante : elle accepte plusieurs variantes d'un role et les transforme en une valeur standard.

7. DashboardController

Le fichier `app/Http/Controllers/DashboardController.php` affiche le tableau de bord principal. Pour le moment, les donnees sont statiques et definies dans le code :

- Indicateurs KPI : revenu total, nombre de SMS+, nombre de services, alertes actives.
- Evolution journaliere et mensuelle des revenus.
- Top 5 des services par revenu.
- Repartition du revenu par fournisseur.
- Liste d'alertes de demonstration.

Ces donnees devront etre remplacees par des requetes Oracle reelles, surtout depuis

`ra_t_agg_mmg` et `ra_t_agg_occ` .

8. CRUD des services

Fichiers principaux :

- `app/Http/Controllers/Operations/ServiceController.php`
- `app/Models/Service.php`
- `resources/views/operations/services`

Cette partie permet de lister, rechercher, ajouter, modifier et supprimer des services. Elle s'appuie sur la table `services` et relie chaque service a un fournisseur :

```
public function provider(): BelongsTo
{
    return $this->belongsTo(VasServiceProvider::class, 'provider_id', 'id');
}
```

Lors de l'ajout, l'ID est calculé avec `max(id) + 1`. Cela vient probablement du fonctionnement Oracle existant sans auto-increment Laravel classique.

9. CRUD des fournisseurs

Fichiers principaux :

- `app/Http/Controllers/Operations/ProviderController.php`
- `app/Models/VasServiceProvider.php`
- `resources/views/operations/providers`

Le fournisseur est stocké dans la table `service_provider`. Sa relation avec les services est :

```
public function services(): HasMany
{
    return $this->hasMany(Service::class, 'provider_id', 'id');
}
```

Avant de supprimer un fournisseur, le code vérifie qu'aucun service n'y est rattaché. Cela évite de casser les relations de données.

10. CRUD des utilisateurs

Le fichier `app/Http/Controllers/Admin/UserController.php` gère l'administration des utilisateurs : liste, recherche, création, modification et suppression.

- L'email doit être unique.
- Le mot de passe est stocké avec `Hash::make`.
- Les rôles sont enregistrés au format Oracle : `ADMIN`, `BUSINESS`, `OPERATION`.
- Un utilisateur ne peut pas supprimer son propre compte connecté.

11. Modeles revenus et alertes

Trois modèles simples représentent des tables Oracle utiles au dashboard et aux traitements :

Modele	Table	Utilisation prevue
RevenueAggregateMmg	ra_t_agg_mmg	Aggregations des revenus ou donnees MMG.
RevenueAggregateOcc	ra_t_agg_occ	Aggregations des revenus ou donnees OCC.
Alert	alerts	Alertes ou incidents.

12. Commandes d'import CDR

Deux commandes Artisan sont disponibles :

- `php artisan cdr:import-mmg`
- `php artisan cdr:import-occ`

Leur fonctionnement general :

- Lecture des fichiers CSV depuis `storage/app/cdr/incoming/mmg` ou `storage/app/cdr/incoming/occ`.
- Insertion des lignes dans une table temporaire comme `ra_t_tmp_mmg`.
- Transfert d'une partie des donnees vers une table detail comme `ra_t_detail_mmg`.
- Si tout reussit, le fichier est deplace dans processed.
- En cas d'erreur, le fichier est deplace dans error et le message d'erreur est affiche.

Les commandes utilisent une transaction. C'est utile : en cas d'erreur, Laravel effectue un rollback et evite de laisser des donnees incompletes.

13. Les vues Blade

La vue principale est `resources/views/layouts/app.blade.php`. Elle contient :

- La structure generale des pages internes.
- Les liens CDN vers Bootstrap et Bootstrap Icons.
- La sidebar.
- La topbar et le bouton de deconnexion.
- Le CSS global des cartes, graphiques, tableaux et adaptations mobiles.

La sidebar est dans `resources/views/partials/sidebar.blade.php`. Son contenu change selon le role actif.

14. Page de connexion

Le fichier `resources/views/auth/login.blade.php` affiche une page independante du layout principal. Elle contient :

- Le choix du type d'utilisateur : Admin, Business ou Operationnel.
- Les champs email et mot de passe.
- Le lien Forget password.
- Les messages d'erreur ou de succes.
- Une note sur le mode demo login.

15. Configuration de la base de donnees

Dans `config/database.php`, la connexion Oracle est definie ainsi :

```
'oracle' => [  
    'driver' => 'oracle',  
    'host' => env('DB_HOST', '127.0.0.1'),  
    'port' => env('DB_PORT', '1521'),  
    'database' => env('DB_DATABASE', ''),  
    'service_name' => env('DB_SERVICE_NAME', ''),  
    'username' => env('DB_USERNAME', ''),  
    'password' => env('DB_PASSWORD', ''),  
]
```

Pour utiliser Oracle, le fichier `.env` doit contenir des valeurs de ce type :

```
DB_CONNECTION=oracle  
DB_HOST=...  
DB_PORT=1521  
DB_SERVICE_NAME=...  
DB_USERNAME=...  
DB_PASSWORD=...
```

16. Tests actuels

Les tests existants sont simples :

- `tests/Unit/ExampleTest.php` : verifie que true vaut true.
- `tests/Feature/ExampleTest.php` : verifie que `/` redirige vers `/login`.

La commande suivante a ete executee avec succes :

```
php artisan test
```

```
Tests: 2 passed, 3 assertions
```

17. Remarques techniques importantes

- Certains textes francais ont un probleme d'encodage, par exemple `DÃ@connexion` au lieu de `Déconnexion`.
- Le fichier `.env.example` est encore configure par default pour SQLite, alors que l'application vise Oracle.
- Les migrations actuelles ne creent pas toutes les tables applicatives, comme `services`, `service_provider` et `ra_t_*`.
- Le dashboard utilise des donnees statiques. Il faudra le connecter aux vraies donnees Oracle.
- L'utilisation de `max(id) + 1` peut poser probleme si deux utilisateurs creent des lignes en meme temps. Une Oracle Sequence serait plus robuste.
- Bootstrap et Chart.js sont charges depuis des CDN. L'interface depend donc d'internet sauf si ces assets sont installes localement.

18. Plan de travail conseille

- Corriger l'encodage des textes francais dans les Blade et Controllers.
- Mettre a jour `.env.example` avec les variables Oracle et demo login.
- Connecter le dashboard aux tables `ra_t_agg_mmg` et `ra_t_agg_occ`.
- Remplacer `max(id) + 1` par des Oracle Sequences.
- Ajouter des tests pour login, roles et CRUD.
- Transformer les placeholders en vraies pages selon les besoins du projet.

19. Resume rapide

Pour comprendre rapidement le projet, commence par `routes/web.php`, puis lis les Controllers pour la logique, les Models pour les tables, et les vues Blade pour l'interface. Les parties les plus importantes aujourd'hui sont l'authentification, les roles, les CRUD utilisateurs, services et fournisseurs, ainsi que les commandes d'import CDR.

Document prepare a partir de la lecture du code actuel. Derniere verification : les tests existants passent avec `php artisan test`.